Virtual Scrapbook MVP: Technical Deep Dive

This document provides a comprehensive technical overview of the Virtual Scrapbook application, a React-based platform designed for building structured evidence booklets (e.g., immigration evidence) through Markdown and multi-media asset management.

1. Core Architecture

The application is a single-page React application (SPA) that manages state locally without a persistent backend in this MVP phase. It utilizes a **Single-File Mandate** for high performance and immediate feedback.

Tech Stack

- **Framework:** React (Functional components, Hooks).
- **Styling:** Tailwind CSS (Utility-first CSS).
- **Markdown Parsing:** `marked.js` with custom renderers.
- **PDF Processing:** `pdf.js` (Mozilla).
- **Icons:** `lucide-react` .
- **AI Engine:** Google Gemini 2.5 Flash.

2. State Management & Data Model

The application state is centralized in `App.jsx` using several `useState` hooks.

Asset Structure

Assets (Images and PDFs) are stored as a map of objects:

```
{
  "filename.jpg": {
    "name": "filename.jpg",
    "type": "image" | "pdf",
    "data": "...", // Original file
    "thumbnail": "...", // Used for library preview
    "pages": ["..."], // Array of strings (PDFs only)
    "numPages": 1,
    "caption": "",
    "date": "",
    "location": ""
  }
}
```

Note: Large assets are stored as Base64 strings in memory. For production, these would be moved to S3/Cloud Storage.

3. The PDF "Explosion" Pipeline

One of the most complex features is the conversion of multi-page PDFs into a seamless "scrapbook" experience.

1. **Upload:** User drops a PDF.
2. **Initialization:** `pdfjsLib.getDocument` loads the binary data.
3. **Iteration:** The code loops from `1` to `pdf.numPages`.
4. **Rendering:** For each page:
   - A hidden `<canvas>` is created.
   - The PDF page is rendered to the canvas context at a specific `viewport` scale (1.5x for high quality).
   - `canvas.toDataURL('image/jpeg')` converts the canvas to a string.
5. **Storage:** These images are stored in the `pages` array of the asset object.

4. Smart Editor Features

The editor is a "controlled" `textarea` enhanced with custom keyboard and drag-and-drop logic.

List Continuation (Auto-patterns)

In `handleKeyDown`, the app intercepts the 'Enter' key:

- **Checklists:** If the current line starts with `- [ ]`, pressing Enter creates a new line with a fresh `- [ ]`.
- **Bullet Points:** If the line starts with `-`, it continues the bullet.
- **Escape Logic:** If the current line is just a prefix (e.g., `- [ ]` with no text), a second 'Enter' keypress deletes the prefix and exits the list mode.

Drag-and-Drop Caret

When dragging an asset from the library to the editor:

- `handleDragOver` calls `textarea.focus()`.
- While browser-standard textareas don't show a native "drop caret" easily, focusing ensure the user can see where the text cursor currently resides before releasing.

5. Rendering Pipeline & Syncing

The conversion from Markdown to a physical-style booklet follows a multi-step process.

Step 1: Marked.js Customization

We override the default `renderer`:

- **Headings:** Injected with unique IDs (`heading-0`, `heading-1`) to enable the **Document Map** navigation.
- **Checkboxes:** Injected with a `data-checkbox-index` attribute. This index is crucial for the **Checklist Sync**.

Step 2: Custom Tag Interception

The app uses Regex to find custom tags:

- `[[file:filename.jpg]]` : Triggers an image block with captions and metadata.
- `:::pdf file=name.pdf:::` : Triggers the rendering of all stored JPEG pages from the PDF pipeline.

Step 3: Checklist Synchronization

When a user clicks a checkbox in **Render Mode**:

1. The `onClick` handler catches the `data-checkbox-index`.
2. `toggleCheckbox` runs a global regex on the **raw Markdown** string.
3. It counts every instance of `[ ]` or `[x]` and flips the state of the one matching the clicked index.
4. This ensures the source text remains the "Source of Truth."

6. Gemini AI Integration

The app leverages `gemini-2.5-flash-preview-09-2025` for two distinct tasks:

AI Polish (Text-to-Text)

- **Prompting:** Uses a system instruction to act as an "immigration assistant."
- **Output:** Returns a polished version of the user's narrative while strictly preserving Markdown syntax.

Smart Caption (Image-to-Text)

- **Multimodal:** Sends the Base64 image data alongside a prompt.
- **Structured Output:** Uses `responseMimeType: "application/json"` to ensure the model returns a parseable object containing `caption`, `date`, and `location`.

7. Responsive & Print Logic

- **Mobile:** Uses a `activeMobileTab` system to toggle between the full-screen Editor and the Asset Library.
- **Print:** Employs `@media print` CSS.
  - Removes all UI (Header, Sidebars, Toolbars).
  - Forces `.scrapbook-page` to `width: 210mm` (A4).
  - Sets `page-break-after: always` on each segment separated by `---` (horizontal rules) in Markdown.